

PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Prof. Rogerio Mandelli

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

Aula 02 | 06 mai 2026

UVA ●
A sua melhor escolha.

Aula 2 – Projeto e Implementação de Linguagens

Como decisões de projeto impactam:
Desempenho, portabilidade, segurança, produtividade e
manutenção de software

Objetivos

- **Entender** por que linguagens são projetadas de formas diferentes
- **Relacionar** decisões de projeto aos objetivos das linguagens
- **Diferenciar:**
 - compilação
 - interpretação
 - execução híbrida
- **Entender** o papel de:
 - compiladores
 - interpretadores
 - JVM
- **Comparar** C, Python e Java quanto a:
 - desempenho
 - portabilidade
 - produtividade
 - abstração
 - manutenção
- **Introduzir** como implementação afeta:
 - tipos
 - escopo
 - memória
 - semântica





Bloco 1:

INFLUÊNCIAS NO PROJETO

DE LINGUAGENS

#ConhecimentoTransforma

Influências no Projeto de Linguagens

1. **Hardware (máquina)** — quanto o computador consegue fazer.
2. **Modo de pensar (paradigma)** — como o programador organiza o raciocínio.
3. **Facilidade de programar e segurança** — quanto tempo leva e quantos erros evita.
4. **Facilidade de manter o código** — o quão fácil é entender e mudar depois.
5. **Rapidez e onde pode rodar** — velocidade e compatibilidade com diferentes sistemas.
6. **Leitura humana vs leitura da máquina** — código fácil de ler ou fácil de rodar?

Influências no Projeto de Linguagens

1. Hardware (a máquina)

- Cada linguagem nasce de acordo com o que **os computadores da época conseguem fazer**.
- **Antes**: havia **pouca memória e processadores lentos**, então linguagens como **C** davam acesso direto ao hardware.
- **Hoje**: máquinas são muito mais rápidas — isso permitiu linguagens mais “confortáveis”, como **Python**.
- **Exemplo**: um código em **C** precisa de mais detalhes (declarações, ponteiros); em **Python**, basta escrever a lógica.

Influências no Projeto de Linguagens

2. Modo de pensar (paradigma)

- Um paradigma é **um estilo de resolver problemas** com código.
- Alguns focam em **passos e comandos** (C, Python imperativo).
- Outros focam em **funções, objetos ou lógica** (Python funcional, Prolog lógico).
- ***Exemplo:***
 - **Imperativo** → “faça passo a passo”
 - **Funcional** → “diga o que quer que aconteça”

Influências no Projeto de Linguagens

3. Facilidade de programar e segurança

- Linguagens buscam equilibrar **facilidade de uso** e **controle dos erros**.
 - **Python**: ajuda a começar rápido, mas pode deixar passar erros escondidos.
 - **C**: mais exigente: você declara tipos, mas isso **previne muitos problemas**.
- ***Exemplo:***
 - **C**: precisa declarar que x é inteiro.
 - **Python**: tipo muda automaticamente.

Influências no Projeto de Linguagens

4. Facilidade de manter o código

- Não basta fazer funcionar — é preciso que **outra pessoa entenda o código depois**.
- Linguagens com **nomes claros, indentação e organização** facilitam isso.
 - **Python**: muito usado no ensino porque o código “parece texto”.
- ***Exemplo***:
 - `for aluno in lista:` é mais legível que `for (i=0; i<n; i++)` .

Influências no Projeto de Linguagens

5. Rapidez e onde pode rodar (eficiência e portabilidade)

- **C:** muito rápido, mas o programa precisa ser recompilado em cada sistema.
- **Python:** mais lento, mas roda em qualquer computador com interpretador.
- ***Exemplo:*** mesmo programa em **Python** funciona no Windows, Linux e Mac sem mudar nada.

Influências no Projeto de Linguagens

6. Leitura humana vs leitura da máquina

- Linguagens “próximas da máquina” (como **C**) são rápidas, mas **menos intuitivas**.
- Linguagens “próximas do ser humano” (como **Python**) são fáceis de ler, mas **menos rápidas**.
- **Pergunta:** “Você prefere entender o código facilmente ou que ele rode o mais rápido possível?”

Toda linguagem é uma decisão de engenharia

Nenhuma linguagem maximiza simultaneamente:

- desempenho
- simplicidade
- portabilidade
- segurança
- produtividade
- facilidade de manutenção

Projetar uma linguagem significa:

- escolher prioridades
- aceitar limitações
- equilibrar *trade-offs*

Bloco 2:

EVOLUÇÃO DAS LINGUAGENS E AUMENTO DA ABSTRAÇÃO

Por que novas linguagens surgiram?

As linguagens evoluem para resolver limitações das anteriores.

Exemplos:

- **Assembly** → difícil produtividade
- **C** → necessidade de abstração mínima sem perder desempenho
- **C++** → modularização e orientação a objetos
- **Java** → portabilidade e segurança
- **Python** → produtividade e simplicidade

Linha evolutiva da abstração

Assembly → **C** → **C++** → **Java** → **Python**

Comparar:

- abstração
- produtividade
- controle
- dependência do hardware

O que cada geração priorizou?

Linguagem	Principal prioridade
Assembly	Controle total
C	Eficiência
C++	Modularidade
Java	Portabilidade
Python	Produtividade

Bloco 3:

MODELOS DE IMPLEMENTAÇÃO

E EXECUÇÃO

Métodos de Implementação

- **1. Compiladas** → o código vira **programa nativo** antes de rodar
Ex.: C, C++
- **2. Interpretadas** → o **interpretador** lê e executa **na hora linha a linha**
Ex.: Python
- **3. Híbridas (bytecode + VM/JIT)** → compila para **bytecode** e a **JVM** executa/otimiza
Ex.: Java
- **4. Pré-processadores** → tratam **diretivas** antes da compilação
Ex.: #include, #define (C)

Compilação em Linguagens como C

1. Compiladas (C/C++)

- **Ideia:** “tradução completa” antes de rodar.
- **Resultado:** arquivo **.exe/binário** para aquele sistema/CPU.
- **Vantagens:** velocidade, controle fino do hardware.
- **Custo:** precisa **compilar** a cada mudança e **para cada SO/CPU**.
- **Demo:**

```
gcc soma.c -o soma  
./soma
```

gcc = **GNU Compiler Collection**, compilador do C (também para C++).
soma.c = **arquivo-fonte** (seu código em C).
-o soma = diz ao compilador para gerar um **arquivo executável** chamado **soma**.

Etapas da Compilação em C

1. Compiladas (C/C++)

- **Etapas internas que o GCC faz (de forma automática):**
 - **Pré-processa** → resolve diretivas como `#include` e `#define`.
 - **Compila** → traduz o código C para **assembly**.
 - **Monta** → converte assembly em **código de máquina (objeto)**.
 - **Liga (link)** → junta com bibliotecas (`stdio.h`, etc.) e cria o **executável final**.
 - **Resultado**
 - Criação do **arquivo executável** chamado `soma` (ou `soma.exe` no Windows)

Exemplo de Código Compilado — C

Exemplo em C (código compilado)

ex_21_C.c

```
#include <stdio.h>

int main()
{
    int a, b;
    printf("Digite 1o. número: ");
    scanf("%d", &a);
    printf("Digite 2o. número: ");
    scanf("%d", &b);
    printf("Soma = %d\n", a + b);
    return 0;
}
```

Interpretação em Linguagens como Python

2. Interpretadas (Python)

- Ideia: “leitura e execução” linha a linha, **sem gerar .exe**.
- **Vantagens:** facilidade, portabilidade (se houver interpretador).
- **Custo:** velocidade menor; erros podem surgir **em tempo de execução**.
- **Demo:**
`python soma.py`

Exemplo de Código Interpretado — Python

Exemplo em Python (código interpretado)

ex_21_P.py

```
a = int(input("Digite o 1o. número: "))  
b = int(input("Digite o 2o. número: "))  
print('Soma =', a + b)
```

Execução Híbrida em Java

3. Híbridas (Java)

- **1. Código-fonte** em Java (arquivo.java).
- **2. Compilação → Bytecode (.class).** Compilador Java (javac)
 - Traduz o código para uma **linguagem intermediária**, chamada **bytecode** (arquivo.class).
 - .class **não é código de máquina** (não roda direto no processador), mas também **não é o código-fonte original** — é algo no meio do caminho, mais compacto e padronizado.
 - **Bytecode = instruções universais** que qualquer “máquina virtual Java” (JVM) consegue entender.

JVM e Execução do Bytecode

3. Híbridas (Java)

- **3. JVM (Java Virtual Machine)** é o programa responsável por **rodar o bytecode**.
 - JVM **lê** o `bytecode` (`arquivo.class`);
 - **Interpreta** cada instrução;
 - Quando percebe que uma parte do código está sendo repetida várias vezes (por exemplo, dentro de um `for` grande), **compila essa parte em tempo de execução** — isso é o **JIT** (*Just-In-Time Compilation*).
 - Esse **modelo híbrido** influenciou posteriormente:
 - .NET (CLR)
 - V8 JavaScript Engine
 - PyPy

JIT (Just-In-Time Compilation)

3. Híbridas (Java)

- 4. JIT (Just-In-Time Compiler)

- “Just in time” significa “**na hora certa**”: o código é compilado para máquina **durante a execução**, e não antes.
- A JVM identifica **trechos usados com frequência** (“**hot spots**”) e os transforma em código de máquina para rodar mais rápido nas próximas execuções.
- Isso combina o melhor dos dois mundos:
 - início rápido (como Python);
 - velocidade otimizada depois de alguns segundos (como C).
- Essa **arquitetura** permitiu que **Java** se tornasse **dominante** em **aplicações corporativas multiplataforma**.

Componentes do Modelo Java

Termo	O que é	Analogia
Compilador (javac)	Traduz o código Java para bytecode (.class)	“Tradutor universal”
Bytecode	Código intermediário independente do sistema	“Língua comum” entre Java e o computador
JVM	Programa que lê e executa o bytecode	“Intérprete multilíngue”
JIT	Parte da JVM que acelera o código repetido	“Aprende o caminho mais rápido”

Exemplo de Código Híbrido — Java

Exemplo em Java (código híbrido)

ex_21_J.java

```
import java.util.Scanner; // biblioteca para leitura de dados

public class ex_21_J_soma
{
    public static void main(String[] args)
    {
        Scanner entrada = new Scanner(System.in); // cria o leitor de entrada

        int a, b;
        System.out.print("Digite o 1o. número: ");
        a = entrada.nextInt();
        System.out.print("Digite o 2o. número: ");
        b = entrada.nextInt();

        System.out.println("Soma = " + (a + b));

        entrada.close(); // boa prática: fechar o Scanner
    }
}
```

Pré-processadores em C

4. Pré-processadores (C)

- Rodam **antes** do compilador. Incluem cabeçalhos, trocam símbolos, **condicionam código**.
- **Ex.:** `#include <stdio.h>, #define TAM 100, #if/#endif.`

Como Funciona o Pré-processamento

4. Pré-processadores (C)

- Antes do compilador **traduzir o código em C para máquina**, existe uma **etapa anterior** chamada **pré-processamento**.
- **Pré-processador lê e modifica o código-fonte** antes da compilação, seguindo comandos especiais que começam com **#**.

Inclusão e Substituição de Código

4. Pré-processadores (C)

- Executa **três funções principais**:
 - **a) Inclui arquivos externos**
 - Quando detecta `#include <stdio.h>`, **insere o conteúdo desse arquivo** no seu código antes de compilar.
 - Isso traz funções como `printf()` e `scanf()`.
 - Compilador nem enxerga o `"#include"`; identifica tudo pronto.
 - **b) Substitui símbolos definidos**
 - Com `#define`, você cria **atalhos ou constantes**.
 - Exemplo:

```
#define PI 3.14159
```

- Em todo o código, o **pré-processador troca** `PI` por `3.14159`

Compilação Condicional

4. Pré-processadores (C)

- Executa **três funções principais**:

- **c) Condiciona a compilação**

- Com `#if`, `#ifdef`, `#endif`, ele **escolhe trechos de código** para compilar ou ignorar.

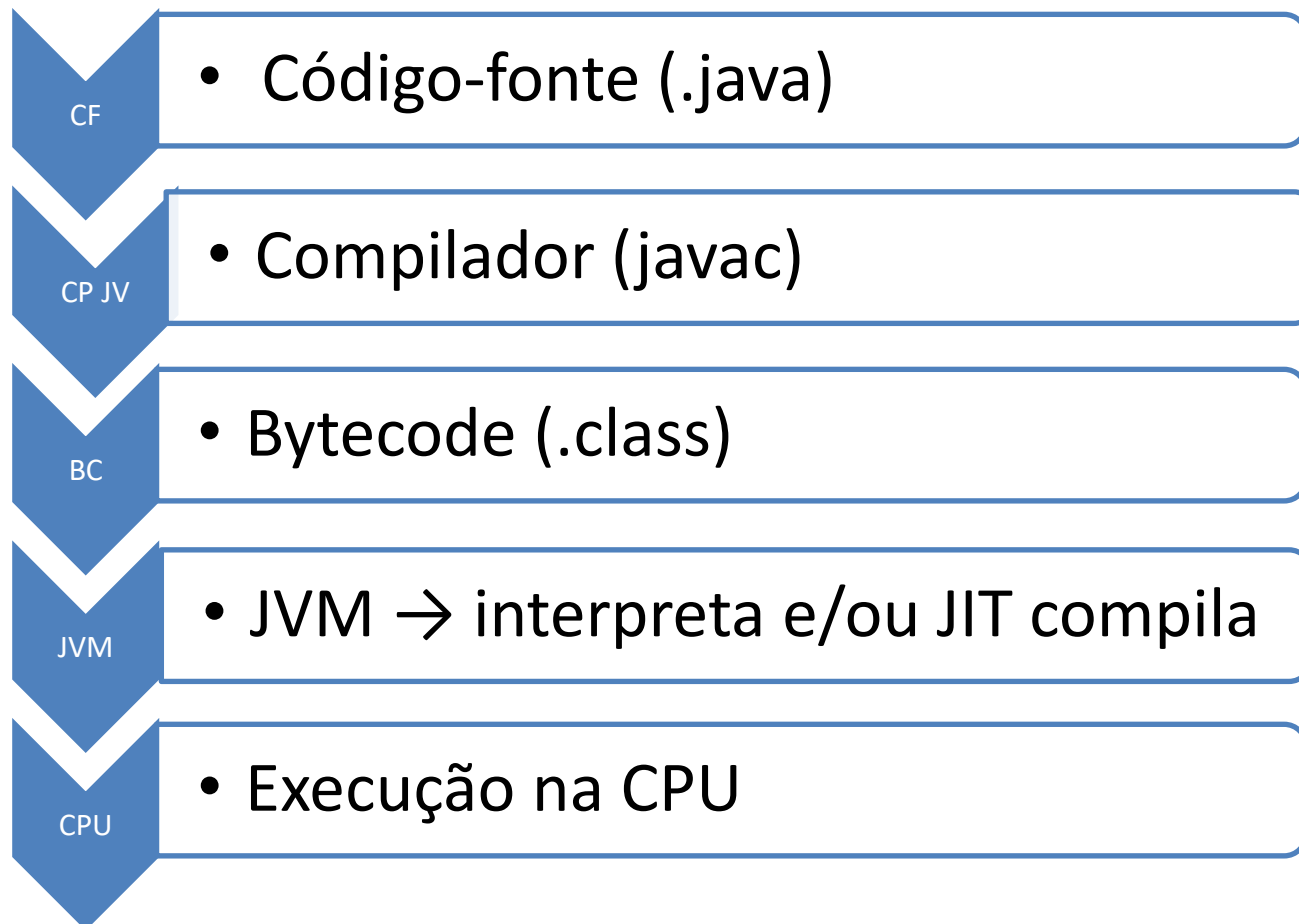
- Exemplo:

```
#define TESTE
#ifdef TESTE
    printf("Modo de teste ativo!\n");
#endif
```

→ Só compila essa linha se TESTE estiver definida.

Pipeline de Execução Híbrida — Java

- Híbrido (Java):



Compilação × Interpretação

- Compilação (C):



- Interpretação (Python):





Bloco 4:

COMPARAÇÃO TÉCNICA

ENTRE LINGUAGENS

#ConhecimentoTransforma

Comparativo: C × Python

Critério	C (Compilado)	Python (Interpretado)
Velocidade	Alta	Menor
Portabilidade	Menor	Alta
Deteccção de erros	Compilação	Execução
Tipagem	Estática	Dinâmica

Métodos de Implementação

Aspecto	Compilada (C)	Interpretada (Python)	Híbrida (Java)
Como roda	Código-fonte é compilado em binário nativo (executável <code>.exe</code> , <code>.out</code>)	O interpretador executa diretamente o código-fonte linha a linha	O código é compilado para bytecode , executado pela JVM
Velocidade	Alta	Menor	Média / Alta (com JIT)
Portabilidade	Baixa (recompilar p/ cada SO)	Alta (mesmo script roda em qualquer SO)	Alta (JVM presente em várias plataformas)
Detecção de erros	Na compilação	Durante a execução	Em ambos (compilação e execução)
Tipagem	Estática (<code>int</code> , <code>float</code> , etc.)	Dinâmica (tipo inferido em tempo de execução)	Estática (declaração explícita de tipos)
Extensão	<code>.c</code>	<code>.py</code>	<code>.java</code>
Comando para executar	<code>gcc soma.c -o soma</code> <code>./soma</code>	<code>python soma.py</code>	<code>javac soma.java</code> <code>java soma</code>
Uso típico	Sistemas embarcados, SOs, drivers	Scripts, ciência de dados, ensino	Aplicações multiplataforma, mobile, corporativas

Ambientes de Programação

- **Programar** envolve mais do que escrever código.
- Um **ambiente moderno** inclui:
 - editor/IDE
 - compilador
 - interpretador
 - bibliotecas
 - *debugger*
 - *build system*
 - *runtime*
 - terminal

Toolchain: C × Python × Java

- **C**
 - VS Code → GCC → Linker → Executável
- **Python**
 - VS Code → Python Interpreter → Execução
- **Java**
 - IDE → javac → Bytecode → JVM

Trade-offs entre linguagens

Critério	C	Python	Java
Desempenho	Alto	Baixo	Médio/alto
Portabilidade	Média	Alta	Muito alta
Produtividade	Média	Alta	Média
Controle memória	Alto	Baixo	Médio
Segurança	Baixa	Média	Alta
Curva aprendizado	Média	Baixa	Média

Escolhas reais de mercado

C	Python	Java
<i>firmware</i>	IA	backend
SO	ciência de dados	Android
<i>drivers</i>	automação	ERP

Bloco 5:

INTRODUÇÃO À SEMÂNTICA, ESCOPO E MEMÓRIA

Introdução à Semântica e Escopo

- Amarração (*binding*): ligação entre identificadores e valores.
- Pode ocorrer em tempo de compilação ou execução.
- Escopo: região onde um identificador é válido.
- Escopo léxico (C, Python) e escopo dinâmico (mais raro).

Introdução à Semântica e Escopo

1. Amarração (*binding*)

- “Ligação entre um **nome (identificador)** e **alguma coisa que ele representa** — um valor, um tipo, uma função, um espaço de memória.”
- *Exemplo em C:*

```
int x = 10;
```
- Nome `x` foi **amarrado** (ligado) a uma posição na memória que contém o valor `10`.
- Essa amarração também define que `x` é do **tipo inteiro**.

Introdução à Semântica e Escopo

2. Escopo

- **Região do programa** onde uma **variável** pode ser **usada** e é **reconhecida**.”
- *Exemplo em C:*

```
int x = 10;  
void funcao()  
{  
    int y = 5;  
}
```

- **x** → variável **global** (visível em todo o programa)
- **y** → variável **local** (só existe dentro da função funcao())

Introdução à Semântica e Escopo

4. Tipos de Escopo

Tipo	O que significa	Linguagens que usam
Escopo léxico (estático)	Local da declaração determina onde o nome é válido.	C, Python, Java, quase todas as modernas
Escopo dinâmico	Valor depende de quem chamou a função (não de onde foi escrita).	Linguagens antigas como Lisp, e alguns casos específicos

- > Atualmente, **quase todas as linguagens modernas** usam **escopo léxico**

Introdução à Semântica e Escopo

4. Tipos de Escopo

Conceito	Python	C
Escopo léxico	Determinado pelo local onde a variável é declarada	Determinado pelo bloco { }
Variável local	Criada dentro de função, só existe lá	Criada dentro de função/bloco
Variável global	Criada fora de função, acessível em todo o programa	Idem
Sombreamento	Um nome local esconde o global de mesmo nome	Igual

Gerenciamento de Memória

O que acontece quando criamos **variáveis**?

- **Variáveis** e **objetos** possuem **tempo de vida** associado ao **modelo de memória** da **linguagem**.
- Algumas linguagens exigem **cuidado manual** (C) e **outras não** (Python).
- Isso afeta **desempenho**, **segurança** e **facilidade** de uso.

→ **C**: alocação manual e estática (`malloc`, `free`)

→ **Python**: alocação automática (`garbage collector`)

Como implementação afeta semântica?

As **decisões** de implementação **influenciam**:

- tipos
- escopo
- gerenciamento de memória
- segurança
- desempenho
- comportamento do programa

Preparação para a próxima aula

Próxima aula:

- *binding*
- tipos
- escopo
- variáveis
- constantes
- memória
- semântica de execução



Bloco 6:

VISÃO GERAL DOS PARADIGMAS

DE PROGRAMAÇÃO

#ConhecimentoTransforma

Paradigmas de Programação

Um paradigma de programação representa:

- uma **forma** de **organizar** programas;
- um **modelo** de **resolução** de problemas computacionais;
- uma **maneira** de **estruturar**:
 - dados
 - controle
 - execução
 - abstrações

Diferentes paradigmas:

- priorizam **diferentes estratégias** computacionais;
- influenciam:
 - legibilidade
 - modularidade
 - reutilização
 - manutenção
 - desempenho
 - concorrência

Uma mesma linguagem pode suportar múltiplos paradigmas

Principais Paradigmas de Programação

Paradigma	Ideia central
Imperativo	sequência de comandos e mudança de estado
Estruturado	organização do fluxo usando sequência, seleção e repetição
Orientado a Objetos	modelagem baseada em objetos e encapsulamento
Concorrente	múltiplos fluxos de execução simultâneos
Funcional	composição e avaliação de funções
Lógico	inferência baseada em regras e fatos
Declarativo	descrição do que deve ser obtido
Orientado a Eventos	execução guiada por eventos externos

Paradigma Imperativo e Estruturado

No **paradigma imperativo**:

- o programa é descrito como uma sequência de instruções
- variáveis mudam de estado ao longo da execução

O **paradigma estruturado** organiza o fluxo utilizando:

- sequência
- seleção
- repetição
- subprogramas/funções

Características:

- controle explícito do fluxo
- forte dependência de estado
- execução passo a passo

Exemplos:

- C
- Pascal
- Python
- Java

Paradigma Orientado a Objetos

No paradigma orientado a objetos:

- programas são organizados em objetos;
- objetos combinam:
 - dados
 - comportamento

Conceitos principais:

- classe
- objeto
- encapsulamento
- herança
- polimorfismo

Objetivos:

- modularidade
- reutilização
- abstração
- manutenção

Exemplos:

- Java
- Python
- C++

Paradigma Funcional e Lógico

Paradigma Funcional

Baseado em:

- funções matemáticas
- composição
- redução do estado mutável

Características:

- funções como elementos centrais
- maior previsibilidade
- facilita paralelismo e processamento distribuído

Exemplos:

- Haskell , Lisp , Scala

Paradigma Lógico

Baseado em:

- fatos
- regras
- inferência lógica

O programa descreve **relações lógicas** e o sistema deduz resultados.

Exemplo:

- Prolog

Paradigma Concorrente e Orientado a Eventos

Paradigma Concorrente

O programa executa:

- múltiplas tarefas
- processos
- threads
- rotinas assíncronas

Objetivos:

- paralelismo
- desempenho
- responsividade

Usado em:

- servidores
- sistemas operacionais
- aplicações distribuídas
- sistemas de tempo real

Paradigma Orientado a Eventos

O fluxo do programa depende de:

- eventos externos;
- ações do usuário;
- mensagens;
- interrupções.

Exemplos:

- interfaces gráficas;
- aplicações web;
- sistemas móveis.

C, Python e Java são Linguagens Multiparadigma

Linguagem	Paradigmas predominantes
C	Imperativo, estruturado
Java	OO, imperativo, concorrente, eventos
Python	Imperativo, OO, funcional

Linguagens modernas frequentemente combinam:

- diferentes paradigmas
- diferentes modelos de execução
- diferentes estratégias de abstração

Isso aumenta:

- flexibilidade
- reutilização
- adaptabilidade



Bloco 7:

EXERCÍCIOS E CONSOLIDAÇÃO

#ConhecimentoTransforma

Exercício 2.2— Escolha de Linguagem de Programação

Uma **empresa de tecnologia** está avaliando **qual linguagem utilizar** em diferentes projetos. Considere apenas as linguagens:

- C
- Python
- Java

Para cada situação a seguir (próximo slide):

- escolha a linguagem mais adequada.
- justifique tecnicamente sua resposta considerando:
 - desempenho
 - portabilidade
 - produtividade
 - facilidade de manutenção
 - acesso ao hardware
 - segurança
 - ambiente de execução
 - paradigma predominante
 - nível de abstração da linguagem

Exercício 2.2— Escolha de Linguagem de Programação

Situação A — Sistema Embarcado

Uma equipe irá desenvolver o software de um microcontrolador presente em um equipamento industrial.

O sistema possui:

- pouca memória RAM
- necessidade de resposta rápida
- comunicação direta com sensores e dispositivos eletrônicos

→ Qual linguagem seria mais adequada?

Situação B — Inteligência Artificial

Uma startup deseja criar rapidamente protótipos de modelos de IA para:

- análise de dados
- treinamento de redes neurais
- automação de tarefas

O foco principal é:

- rapidez de desenvolvimento
- facilidade de experimentação
- disponibilidade de bibliotecas prontas

→ Qual linguagem seria mais adequada?

Situação C — Sistema Bancário

- Um banco desenvolverá um sistema corporativo que:
- roda em diferentes sistemas operacionais;
- possui milhares de usuários simultâneos;
- exige alta segurança;
- precisa de manutenção contínua por muitos anos.

→ Qual linguagem seria mais adequada?

Situação D — Automação Simples

- Um funcionário deseja criar rapidamente um programa para:
- organizar arquivos;
- ler planilhas;
- enviar relatórios automaticamente por e-mail.
- O objetivo é criar a solução com poucas linhas de código e rapidez.

→ Qual linguagem seria mais adequada?

Exercício 2.3— Identificação de erros

Analise os trechos de código a seguir (próximo slide) e **classifique cada situação** como:

- erro de compilação
- erro de execução
- erro semântico/lógico

Além disso:

- explique o motivo
- indique em que momento o problema ocorre

Exercício 2.3— Identificação de erros

TRECHO A — C

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int x = "10";
6      printf("%d\n", x);
7      return 0;
8  }
```

TRECHO B — Python

```
1  x = 10
2  y = 0
3
4  print(x / y)
5
```

TRECHO C — Python

```
1  idade = 20
2
3  if idade > 60:
4      print("Maior de idade")
5  else:
6      print("Menor de idade")
7
```

TRECHO D — Java

```
1  public class Teste
2  {
3      public static void main(String[] args)
4      {
5          int x;
6          System.out.println(x);
7      }
8  }
```

Exercício 2.4— Pipeline de execução

Explique como ocorre o **processo de execução** de programas em:

- C
- Python
- Java

Para **cada linguagem**:

1. indique as etapas principais
2. explique quem executa o código
3. informe quando erros podem ser detectados
4. relacione o modelo com:
 - desempenho
 - portabilidade

Exercício 2.5— Fila Simples C, Python e Java

Considere uma **fila simples de atendimento** com três clientes:

Ana, Bruno, Carla

O programa deve:

- inserir os três clientes na fila
- mostrar quem será atendido primeiro
- remover o primeiro cliente
- mostrar quem ficou na fila

Depois, compare C, Python e Java quanto a:

- quantidade de estrutura obrigatória
- legibilidade
- nível de abstração
- facilidade para representar a fila

Exercício 2.5— Fila Simples C, Python e Java

main.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char fila[3][20] = {"Ana", "Bruno", "Carla"};
6
7      printf("Primeiro da fila: %s\n", fila[0]);
8
9      printf("Cliente atendido: %s\n", fila[0]);
10
11     printf("Fila atual:\n");
12     printf("%s\n", fila[1]);
13     printf("%s\n", fila[2]);
14
15     return 0;
16 }
17
```

Exercício 2.5— Fila Simples C, Python e Java

```
1  fila = []
2
3  fila.append("Ana")
4  fila.append("Bruno")
5  fila.append("Carla")
6
7  print("Primeiro da fila:", fila[0])
8
9  atendido = fila.pop(0)
10 print("Cliente atendido:", atendido)
11
12 print("Fila atual:", fila)
13
```

Exercício 2.5— Fila Simples C, Python e Java

```
FilaSimples.java :
1 import java.util.LinkedList;
2 import java.util.Queue;
3
4 public class FilaSimples {
5     public static void main(String[] args) {
6         Queue<String> fila = new LinkedList<>();
7
8         fila.add("Ana");
9         fila.add("Bruno");
10        fila.add("Carla");
11
12        System.out.println("Primeiro da fila: " + fila.peek());
13
14        String atendido = fila.poll();
15        System.out.println("Cliente atendido: " + atendido);
16
17        System.out.println("Fila atual: " + fila);
18    }
19 }
```

Síntese da Aula

Linguagens de programação são resultado de **decisões de engenharia**

- Diferentes modelos de implementação impactam:
 - desempenho
 - portabilidade
 - segurança
 - produtividade
- C, Python e Java representam diferentes estratégias de abstração e execução.
- Paradigmas influenciam:
 - organização do código
 - modularidade
 - concorrência
 - reutilização
- Implementação afeta diretamente:
 - tipos
 - escopo
 - memória
 - semântica do programa

